Toward Automated Software Requirements Classification

Hala Alrumaih
Information Systems Department
Al Imam Mohammad Ibn Saud Islamic University
Riyadh, Saudi Arabia
haalrumaih@imamu.edu.sa

Abdulrahman Mirza[1], Hessah Alsalamah[2]
Information Systems Department
King Saud University
Riyadh, Saudi Arabia
[1]amirza@KSU.EDU.SA, [2]halsalamah@KSU.EDU.SA

***Abstract*—With the growing awareness of the effects of requirements in software processes, requirements engineering is increasingly becoming an area of focus in software engineering research. A vast number of studies assert that failure in understanding and classifying requirements are the main causes of exceeding costs and allocated time, which in turn results in project failure. Successful software systems development requires consistent and classified requirements. Requirements classification represents an early but critical phase in the requirements analysis stage. While the literature draws a distinction between different types of requirements, in practice it is not always easy to identify such differences. This paper provides an overview of requirements classification, presents some of the existing research studies on requirements classification, and discusses their limitations to yield suggestions for improvement.**

Keywords-software engineering; requirements engineering; requirements classification; artificial intelligence; machine learning

I. INTRODUCTION

In the software engineering area, there is a growing and continuous request to produce a software system in shorter cycles with higher productivity and quality [1]. This request mainly depends on how well a software system fulfills the needs of its environment and its users. Software requirements comprise these needs through requirements engineering processes, which are among the most important parts in software engineering. Successful development of software systems requires complete, consistent and classified requirements.

Accurate requirements classification is always a critical factor in any project's success. Research [2], [3], [4] has shown the risks generated when working with incorrect ways to classify requirements. These risks can even entail project failure due to projects going over time or over budget. At the very least, projects would expend extra efforts by the time of completion. Reports show that 71% of software failures are due to the lack of clear understanding and classifying of the requirements [5]. From the Standish Group, CHAOS report [6] stated the reasons for IT project failure in the USA. It showed that IT project success rates were only 16.2%; the others ended in failure. The second factor that cause project challenges is incomplete requirements and specification. The quality of requirements specification has an important impact on the outcome of the project due to poor and imprecise requirements classification. Another survey shows that IT projects pay for poor requirements classification [7].

II. REQUIREMENTS ENGINEERING

Software engineering consists of many subfields. Requirements engineering is one of the important subfields of software engineering. Successful requirements engineering involves eliciting the needs of users, customers, and other stakeholders; analyzing the stakeholders' requirements, documenting the requirements as software requirements specification; validating that the documented requirements match the stakeholders' requirements, and managing the requirements evolution [8]. Requirements engineering has now changed from being the initial stage in the software development lifecycle to a basic activity that spans across the entire software development lifecycle in many organizations [9].

Requirements development is subdivided into elicitation, analysis, specification, and verification activities. Some researchers consider requirements analysis as the hardest part in the requirements engineering process for developing a software system because it is responsible for deciding what to build [10]. Requirements analysis covers many important stages before producing a high-quality specification document. Its objective is to lead to an agreement between different stakeholders' views [11]. Requirements classification is the first and complicated stage in requirements analysis. This importance comes from diversity of requirements that can be varied in their purpose and in the kinds of properties they represent. There are many effective classifications techniques dedicated to many pioneers in the field of requirements engineering such as Sommerville's technique [12]. This diversity is because of the rich nature of software requirements and depending on the requirements definition methods, and the architecture and design methods being applied for the developed software system.

III. REQUIREMENTS CLASSIFICATION

A. Definition and Overview

From the requirements engineering viewpoint, classification is the arrangement of software requirements into different classes so they could provide meaningful support for decision making and for further analytical processes [13]. These processes may include identification of critical and primary

978-1-5386-4110-1/18/$31.00 ©2018 IEEE

requirements, evaluation of conflicting requirements, and summarizing requirements.

Requirements can be classified on a number of scopes. Examples include [14]:

- Whether the requirement is functional, which means what the system will do, or nonfunctional what means a constraint on the types of solutions that will meet the functional requirements (e.g., accuracy, performance, security and modifiability).
- Whether the requirement is a primary requirement that relay directly on the system by a stakeholder or some other resources or derived from one or more high-level requirements.
- Whether the requirement is on the product or the process.
- Whether the requirements are related to business goals called goal-level requirements, related to the problem areas called domain-level requirements or related to what to build called design-level requirements.
- The requirement priority. In general, a higher priority is the more essential requirement in meeting the overall goals of the system. Classification process often uses a fixed-point scale such as mandatory, highly desirable, desirable, or optional.
- The scope of the requirement. A requirement's scope refers to the range to which a requirement affects the system and its components. Some requirements, specifically nonfunctional ones, have a universal scope that cannot be allocated to a discrete component. Hence, a requirement with universal scope may strongly affect many components, one with a limited scope may have little effect on the satisfaction of other requirements.
- Volatility/stability. Some requirements will change during the lifecycle of the software and even during the development process itself. This is useful if the software engineer estimates the likelihood of a requirement changing.

B. *Importance of Requirements Classification*

In the primary phase of requirements engineering, classifying the requirements in different groups may support further activities much easier than direct working [13].

In the software engineering domain, classification has been an attractive keyword from last couple of decades for classifying requirements, errors, software features, software risks, and software testing [15]. From the requirements engineering point of view, classification can play very helpful roles in many ways such as improving the understandability of user requirements, priorities setting, and assessment of their quality [16]. Moreover, requirements classification can also be used to reduce the difficulty of decision making by reducing a large number of requirements into fewer groups [17].

The following are the major reasons for making these classifications [18]:

First, there is a difference between functional requirements and nonfunctional requirements. This is because there are usually much more challenging problems in designing and testing for nonfunctional requirements. Obtaining a design to meet the nonfunctional requirements often takes more time and is filled with complex problems. Functional requirements appear to be more straightforward.

Second, it is useful to describe constraints separately. By having the separate word 'constraints', there will be a limit in the design space explicitly without confusing the difference between function space and design space. For example, if the function is "develop an online game," the customer probably wants the game to work with any browser, without the need for a plugin, rather than implementing a plugin for each browser.

IV. Requirements Classification Techniques

Software requirements reflect the facilities and services that the system should provide. These facilities and services could be very essential in a way that they become core to the system and not to be ignored, where other facilities and services could be optional to the system. Consequently, software requirements could be essential, which represents services that must be met, desirable, which represents services that are useful but not necessary to be in the system, or optional, which are requirements that could be excluded from the system [19].

Most software practitioners just use the 'requirements' concept. Instead of that, many software requirements classifications provide clear priorities for the requirements in the system. Software engineering scientists handle this need by providing several requirements classifications techniques. Three main classifications dedicated to three pioneers in the field of requirements engineering now follow.

A. *Ian Sommerville's Classification*

Sommerville [12] distinguished between the different levels of requirements descriptions by using the term 'user requirements' to mean the high-level abstract requirements and 'system requirements' to mean the detailed description of what the system should do. He wrote requirements at different levels of detail because different readers use them in different ways as illustrated in Fig. 1.

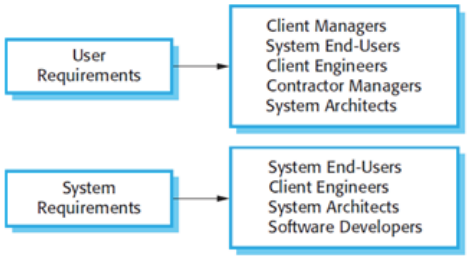

Figure 1. Different Levels of Requirements Description [12]

In his book, Sommerville defined user requirements and system requirements as follows.

- User requirements describe what services the system is expected to deliver to system users and the constraints or limitations under which it must operate.
- System requirements describe the software system's functions, services, and operational constraints in more details.

Normally, Sommerville classified software system requirements as functional requirements or nonfunctional requirements:

- Functional requirements refer to how the system should react to particular inputs, and how the system should behave in particular situations.
- Nonfunctional requirements refer to constraints on the services or functions offered by the system. They include scheduling or timing constraints, constraints on the development and implementation process, and constraints imposed by standards.

He also mentioned that the nonfunctional requirements may come from required characteristics of the software called product requirements, the organization developing the software called organizational requirements, or from external causes.

- Product requirements describe the behavior of the software. Examples include security requirements, and performance requirements.
- Organizational requirements are broad system requirements derived from policies and procedures in the customer's and developer's organization. Examples include environmental requirements that identify the working environment of the system and operational process requirements.
- External requirements cover all requirements that are derived from factors external to the system and its development process. These may include regulatory requirements that defined what one must do to the system to be approved for use.

Fig. 2 shows Sommerville's classification of nonfunctional requirements.

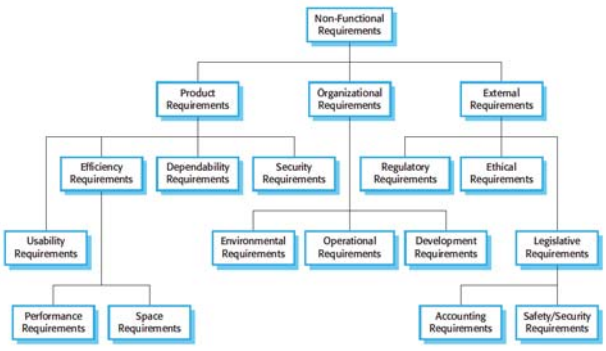

Figure 2. Types of Nonfunctional Requirements [12]

B. *Soren Lauesen's Classification*

Lauesen [20] classified software requirements based on what the requirements specification should contain and what the software should do, namely, the goal-design level. There are various requirements that the requirements specification should contain as Lauesen explained in his book.

- Data requirements: It is an important part of the requirements to specify: What data should the system input and output use? What data should the system store internally?
- Functional requirements specify the tasks of the system, how it reports, computes, changes, and transfers data.
- Quality requirements specify the satisfactory way by which the system performs its intended functions. They are also called nonfunctional requirements.
- Managerial requirements are in a gray area between requirements and contractual issues. There is a need to know when requirements will be delivered, the price and when to pay it, how to verify that everything is working, what happens if things go wrong.

On the other hand, Lauesen presented four possibilities for the requirements based on the goal-design level. These include:

- Goal-level requirement specifies a business goal that can be verified, although only after some period of operation.
- Domain-level requirement includes the requirements going on around the product.
- Product-level requirement specifies what should come in and out of the product.
- Design-level requirement specifies the product interfaces in detail without showing how to implement it inside the product.

C. *Karl E. Wiegers's Classification*

As illustrated in Fig. 3, Wiegers [21] gave a better overview of what information one needs to elicit, analyze, specify, and validate software requirements.

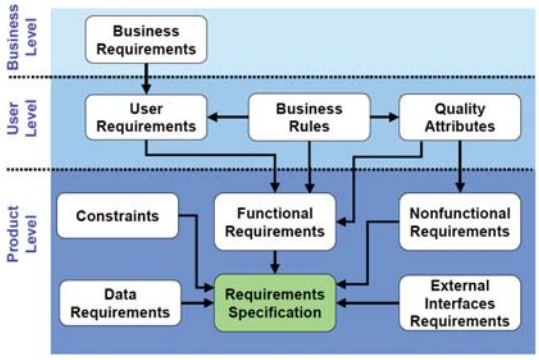

Figure 3. Wiegers's Requirements Classification [21]

Business requirements define the business difficulties to be solved or the business chances to be addressed by the software product. In general, the business requirements explain why the software product is being developed. Business requirements are typically specified in terms of the purposes of the customer or organization requesting the development of the software.

User requirements specifies the functionality of the software product from the user's perspective. They define what the software must to do for the users to accomplish their objectives. Multiple user level requirements may be needed to fulfill a single business requirement.

The product's functional requirements that describe the software functionality should be built into the product to allow users to accomplish their tasks, thereby satisfying the business requirements. Multiple functional level requirements may be needed to achieve a user requirement.

In contrast to the business requirements, business rules are the certain policies, standards, practices, regulations, and guidelines that explain how the users do business and are therefore considered user-level requirements. The software product must follow these rules in order to function appropriately within the user's domain. User-level quality attributes are nonfunctional characteristics that state the software product's quality. Quality attributes (sometimes called the 'ilities') include reliability, availability, security, safety, maintainability, portability, usability, and other properties. A quality attribute may transform into product-level functional requirements for the software that specify what functionality must occur to meet the nonfunctional attribute. A quality attribute may also be interpreted as product-level nonfunctional requirements that state the features the software must produce in order to meet that attribute.

The external interface requirements explain the requirements for the information movement across shared interfaces to hardware, users, and other software applications outside the borders of the software product being developed. The constraints define any restrictions executed on the choices that the supplier can make when designing and implementing the software. The data requirements define the precise data entries or data structures that should be included as part of the software product.

V. Using Artificial Intelligence Techniques In Requirements Engineering

There are some encouraging results that have been produced from the application of artificial intelligence techniques in requirements engineering. Some of the successful artificial intelligence techniques include knowledge-based approach, automated reasoning, expert systems, heuristic search strategies, and machine learning. Machine learning methods have participated in the software development life cycle. They offer a viable alternative to automate many software engineering issues.

Zhang [22] showed how machine learning algorithms can be used in dealing with software engineering tasks problems. Machine learning algorithms can be used to implement tools for requirement engineering which holds an important area in software development and maintenance tasks. In addition, these algorithms are integrated into software products to make them adaptive and self-configuring.

Scott and Cook [23] proposed the structure of an intelligent assistant used in requirements elicitation and assessment. They presented the attributes needed to define the qualities of good requirements and requirements sets. This work was a rigorous attempt to identify and demonstrate a reasoning strategy, which delivers significant cognitive support for requirements elicitation and evaluation.

In military command centers, Lange [24] discussed the use of machine learning and text classification technologies to support the knowledge management requirements. Military communities are starting to use blogs and related tools to share information and provide a comparable environment for the use of blogs in system requirements discussions. In large distributed command and control enterprises, the machine learning tools offer many benefits to requirements engineering. For example, the machine learning tools work well if there is a need to extract information from a large diverse population of stakeholders using network communication tools like blogs.

Based on machine learning theory, Dong, Yang, and Wang [25] proposed a novel approach to generate requirements for a domain ontology evolution automatically in the light of P2P network routing model and storage characteristics of ontology resource. The method applies a comprehensive method considering term frequency, term domain, similarity with original ontology, and other factors to extract key concepts from texts. Based on using a Naïve Bayes classifier, it matches those key concepts with original ontology and extracts their relationships.

Del Sagrado, Del Águila, and Orellana [26] proposed an architecture to integrate between knowledge engineering and requirements. This work presented the architecture for the seamless integration of Computer-Aided Requirements Engineering (CARE) tools to manage requirements using some AI techniques such as Bayesian networks. A Bayesian is used in the requirements validation task in order to validate the software requirements specification of a software development project.

Ammar, Abdelmoez , and Hamdi [27] provided a survey on the application of AI approaches to the software engineering processes which covers the development activities of requirements engineering, software architecture design, and coding and testing processes. The paper reported that these approaches are useful to reduce the time for marketing and improve the quality of software systems. The authors used the terminology and the processes defined by the IEEE 12207 standard of software engineering. On the other hand, they carried out a mapping of current AI techniques. These AI techniques attempt to automate or semi-automate activities and design optimal or semi-optimal solutions in much less time.

Wang [28] proposed an approach to analyze software requirements specifications (SRSs) and extract the semantic information automatically. This approach uses machine learning and ontology. First of all, semantic frames were designed for some common verbs after they calculated from SRS documents in the e-commerce domain. Based on these semantic frames, sentences from SRSs were chosen and labeled manually. The

labeled sentences were used as training data in the machine learning stage. Besides the training data, the approach uses external ontology knowledge to reduce the effect of the data sparsity problem and get reliable results. Experimental results showed that this new approach is effective when it is used in requirements analysis automatically.

It is difficult for requirements engineers to elicit and analyze security requirements clearly. As a result, they are not able to produce the software free from threats and vulnerabilities. Jindal, Malhotra, and Jain [29] intended to mine the descriptions of security requirements present in the SRS document and thereafter develop the classification models. The security-based descriptions are analyzed using text mining techniques and classified into four types of security requirements using decision tree method. The proposed framework can be used by the software engineers and developers in classifying security requirements during early phases of software development. The result analysis showed that all the models have performed very well in predicting their respective type of security requirements.

Requirements engineers mainly document the results of the requirements engineering process using natural language requirements specifications. Besides the real requirements, these documents contain additional explanations, references, summaries, or figures. For the later use of requirements specifications, it is important to be able to differentiate between legally relevant requirements and other information. Therefore, there is a demand to label manually each content element of a requirements specification as "requirement" or "information" by the requirements engineers. However, this manual classification task is time-consuming and error-prone. Winkler and Vogelsang [30] presented an approach to classify the content of a natural language requirements specification as "requirement" or "information" automatically. The approach used the neural networks method to increases the quality of requirements specifications in the sense that it discriminates important content for following activities.

Table I summarizes the previous works, listed by the reference number in the first column. The second column specifies the publication year of the research. The third column states the type of AI technique used in the work. The last column indicates the main objective of using the technique in the work.

VI. Discussion and Recommendations

The literature in research demonstrates that most of the proposed techniques used to classify requirements are either manual techniques, or automated ones with some limitations and gaps. For few requirements, manual classification is used because human experts can deal with the requirements very easily and more accurately. However, it becomes a complex and time-consuming task in projects having many or complex requirements. In such cases, manual classification is suffering from some limitations such as dealing with large numbers of requirements, time-consumption, unavailability of experts, or wrong judgment. In other cases which used automated techniques, there is a need to improve the effectiveness of the developed method to increase the quality of results. Automation can save time and effort as well as decreasing the cost of needed experts' time.

Moreover, using individual classifiers is not sufficient to classify requirements with high performance. Combining different classification techniques can improve classification accuracy. The concept of hybrid scheme that combines more than one classification technique is proposed as an ideal direction for the improvement of the performance of individual classifiers.

This work suggests a different approach in dealing with requirements classification. A hybrid approach using artificial intelligence techniques such as machine learning algorithms is instrumental to classify requirements automatically. The selected algorithms should be chosen from different categories of the developed machine learning algorithms for classification to increase the quality of the result. The concept of hybrid that combines more than one artificial intelligence technique for classification suggests a new aim for the improvement of the performance of individual classifiers. In the context of combining multiple classifiers for classification, some researchers have presented that combining different classification techniques can improve classification accuracy. This approach would minimize the ambiguities that currently exist and would increase the efficiency of software development.

TABLE I. A List of Recent Works Using AI Techniques in Requirements Engineering

Reference	Publication Year	AI technique	Main Objective of using AI technique
[22]	2000	Machine learning	implementing tools for software development and maintenance tasks
[23]	2003	Automated reasoning	developing an intelligent assistant used in requirements elicitation and assessment
[24]	2008	Machine learning	supporting the knowledge management requirements in military command centers
[25]	2011	Naïve Bayes	generating requirements for a domain ontology evolution automatically
[26]	2011	Naïve Bayes	integrating between knowledge engineering and requirements
[27]	2012	AI Techniques	demonstrating the application of AI approaches to the software engineering processes.
[28]	2016	Machine learning	analyzing SRSs and extract the semantic information automatically
[29]	2016	Decision tree	classifying the security-based descriptions into four types of security requirements
[30]	2016	Neural networks	classifying automatically the content of a natural language requirements specification as "requirement" or "information"

VII. Conclusion

The recommendations of this paper intend to benefit practitioners in industry and government as well as academicians in universities and technical institutions of higher learning.

For practitioners, the recommendations would enable industry and government to develop more lucid and concrete requirements classifications that minimize ambiguities. Requirements classification is a critical factor in project success; poor classification may very well cause waste in cost, time, and effort. Using manual techniques to classify requirements takes up great effort and time for requirements engineers. By applying intelligent methods to classify requirements, the process would save much time for them. Moreover, the new process would increase the quality of analyzing requirements, which would produce accurate system specification documents.

For academicians, the indicated recommendations would foster greater need for clearer requirements specifications in software engineering courses and provide an elevated basis for future research in this area.

References

[1] U. Dinger, R. Oberhauser, and C. Reichel, "A Semantic Web Services Approach Towards Automated Software Engineering," in *Web Services, 2006. ICWS'06. International Conference on*, 2006, pp. 935–938.

[2] "Getting requirements right: avoiding the top 10 traps.," IBM Corporation, Software Group, Requirements definition and management, Oct. 2009.

[3] "Strategies for Project Recovery," PM Solutions, 2011.

[4] A. Hussain, E. O. Mkpojiogu, and F. M. Kamal, "The Role of Requirements in the Success or Failure of Software Projects," *International Review of Management and Marketing*, vol. 6, no. 7S, 2016.

[5] C. Lindquist, "Fixing the Software Requirements Mess," *CIO*, 15-Nov-2005. [Online]. Available: https://www.cio.com/article/2448110/developer/fixing-the-software-requirements-mess.html. [Accessed: 29-Sep-2017].

[6] T. Clancy, "The Standish Group Report -CHAOS," Project Smart, 2014.

[7] B. J. E. Powell and 02/07/2008, "IT Pays a Price for Poor Requirements Practices -," *ADTmag*. [Online]. Available: https://adtmag.com/articles/2008/02/07/it-pays-a-price-for-poor-requirements-practices.aspx. [Accessed: 29-Sep-2017].

[8] S. Asghar and M. Umar, "Requirement engineering challenges in development of software applications and selection of customer-off-the-shelf (COTS) components," *International Journal of Software Engineering*, vol. 1, no. 1, pp. 32–50, 2010.

[9] G. Aouad and Y. Arayici, *Requirements engineering for computer integrated environments in construction*. Chichester, West Sussex, U.K. ; Ames, Iowa: Wiley-Blackwell, 2010.

[10] A. O. Sanni, "A new metric for detecting conflict in functional software requirements," *Digital Repository at Iowa State University*, 2006.

[11] P. Haumer, "Requirements engineering with interrelated conceptual models and real world scenes," PhD thesis, Bibliothek der RWTH Aachen, 2000.

[12] I. Sommerville, *Software engineering*, 9th ed. Boston: Pearson, 2011.

[13] N. M. Minhas, S. Majeed, Z. Qayyum, and M. Aasem, "Controlled vocabulary based software requirements classification," in *Software Engineering (MySEC), 2011 5th Malaysian Conference in*, 2011, pp. 31–36.

[14] A. Aurum and C. Wohlin, Eds., *Engineering and managing software requirements*. Berlin: Springer, 2005.

[15] Z. T. Bieniawski, *Engineering rock mass classifications: a complete manual for engineers and geologists in mining*. New York, civil, and petroleum engineering: John Wiley & Sons, 1989.

[16] M. Daneva and A. Herrmann, "Requirements Prioritization Based on Benefit and Cost Prediction: A Method Classification Framework," 2008, pp. 240–247.

[17] M. Aasem, M. Ramzan, and A. Jaffar, "Analysis and optimization of software requirements prioritization techniques," presented at the International Conference on Information and Emerging Technologies, Karachi, 2010, pp. 1–6.

[18] E. Hochmüller, *Requirements classification as a first step to grasp quality requirements*. Proc. Third International Workshop on Requirements Engineering: foundation of Software Quality (REFSQ'97), 1997.

[19] S. L. Pfleeger and J. Atlee, *Software engineering: Theory and practice*. Pearson 2009-02-27, 2006.

[20] S. Lauesen, *Software requirements: styles and techniques*. Harlow: Addison-Wesley, 2002.

[21] K. E. Wiegers and J. Beatty, *Software requirements*, Third edition. Redmond, Washington: Microsoft Press, s division of Microsoft Corporation, 2013.

[22] D. Zhang, "APPLYING MACHINE LEARNING ALGORITHMS IN SOFTWARE DEVELOPMENT," in *The Proceedings of 2000 Monterey Workshop on Modeling Software System Structures*, 2000.

[23] W. Scott and S. C. Cook, "An Architecture for an Intelligent Requirements Elicitation and Assessment Assistant," in *INCOSE International Symposium*, 2003, vol. 13, pp. 470–479.

[24] D. S. Lange, "Text Classification and Machine Learning Support for Requirements Analysis Using Blogs," in *Monterey Workshop*, 2008, pp. 182–195.

[25] J. Dong, M. Yang, and G. Wang, *A Generation Method for Requirement of Domain Ontology Evolution Based on Machine Learning in P2P Network*. Springer Berlin Heidelberg, 2011.

[26] J. Del Sagrado, I. M. Del Águila, and F. J. Orellana, "Architecture for the use of synergies between knowledge engineering and requirements engineering," in *Conference of the Spanish Association for Artificial Intelligence*, 2011, pp. 213–222.

[27] H. H. Ammar, W. Abdelmoez, and M. S. Hamdi, "Software engineering using artificial intelligence techniques: Current state and open problems," in *Proceedings of the First Taibah University International Conference on Computing and Information Technology (ICCIT 2012), Al-Madinah Al-Munawwarah, Saudi Arabia*, 2012, p. 52.

[28] Y. Wang, "Automatic semantic analysis of software requirements through machine learning and ontology approach," *Journal of Shanghai Jiaotong University (Science)*, vol. 21, no. 6, pp. 692–701, Dec. 2016.

[29] R. Jindal, R. Malhotra, and A. Jain, "Automated classification of security requirements," presented at the International Conference on Advances in Computing, Communications and Informatics (ICACCI), 2016, pp. 2027–2033.

[30] J. Winkler and A. Vogelsang, "Automatic Classification of Requirements Based on Convolutional Neural Networks," in *Requirements Engineering Conference Workshops (REW), IEEE International*, 2016, pp. 39–45.